

Angular Learning Series

Angular Foundations — Quick Overview

What you need to know before Topic 20

Contents

- F1. Angular CLI and Project Structure
- F2. Angular Template Syntax -- The Four Bindings
- F3. Built-in Control Flow -- @if, @for, @switch
- F4. Component Lifecycle Hooks
- F5. @Input() and @Output() -- Component Communication
- Foundation Summary -- The Mental Model
- Topic 20 -- RxJS Subscription Management (full response)

Angular Foundations -- Quick Overview

What you need to know before Topic 20 -- no deep dives, just the mental model

F1. Angular CLI and Project Structure

The Command That Creates Everything

```
ng new my-app --standalone --routing --style=scss

cd my-app

ng serve # runs at localhost:4200
```

Folder Structure -- What Actually Matters

```
my-app/

src/

app/

app.component.ts root component

app.component.html root template

app.config.ts app-wide providers (standalone)

app.routes.ts route definitions

main.ts bootstraps the app

styles.scss global styles

angular.json CLI configuration

tsconfig.json TypeScript config
```

CLI Commands You'll Use Daily

```
ng generate component features/users/user-list # create component

ng generate service features/users/user # create service

ng generate pipe shared/pipes/currency # create pipe

ng build --configuration=production # production build

ng test # run tests
```

One Thing to Know

main.ts bootstraps the app with bootstrapApplication(AppComponent, appConfig). appConfig in app.config.ts is where you register app-wide providers -- router, HttpClient, etc. This replaces the old AppModule.

```
// app.config.ts -- the modern Angular entry point

export const appConfig: ApplicationConfig = {
```

```

providers: [
  provideRouter(routes),
  provideHttpClient(),
  provideAnimations()
]
};

```

F2. Angular Template Syntax -- The Four Bindings

Think of Angular templates as HTML with superpowers. Four binding types cover everything:

```

{{ }} -) Interpolation -) display a value
[ ] -) Property binding -) set a property FROM component
( ) -) Event binding -) listen for an event, run component method
[( )] -) Two-way binding -) both directions at once (forms)

```

```

(!-- Interpolation -- display component data --)
(h1){ { title } }(/h1)

(p){ { user.name.toUpperCase() } }(/p)

(!-- Property binding -- set element property from component --)

(button [disabled]="isLoading")Submit(/button)


content(/div)

(!-- Event binding -- run component method on event --)
(button (click)="onSave()")Save(/button)

(form (submit)="onSubmit()")

(!-- Two-way binding -- for forms (NgModel) --)


(!-- searchTerm updates when user types
input updates when searchTerm changes in component --)

(!-- Combining them -- very common pattern --)


```

```

(button

[disabled]="!form.valid || isLoading"

(click)="onSubmit()"

{{ isLoading ? 'Saving...' : 'Save' }}

)/button)

```

F3. Built-in Control Flow -- @if, @for, @switch

Angular 17+ replaced ngIf and ngFor with cleaner block syntax. You'll see both in codebases -- know both.

```

(!-- @if -- show/hide blocks --)

@if (isLoading) {

(div class="spinner")Loading...(/div)

} @else if (error) {

(div class="error">{{ error }}(/div)

} @else {

(div class="content">{{ data }}(/div)

}

(!-- Old syntax -- still common in existing codebases --)



Loading...(/div)

(ng-template #content)(div){{ data }}(/div)(/ng-template)

(!-- @for -- render a list --)

@for (product of products; track product.id) {

(app-product-card [product]="product" /)

}

@empty {

(p)No products found(/p)

}

(!-- Old syntax --)

(app-product-card


```

```

*ngFor="let product of products; trackBy: trackById"

[product]="product")

(/app-product-card)

(!-- @switch -- multiple conditions --)

@switch (user.role) {

@case ('admin') { (app-admin-panel /) }

@case ('editor') { (app-editor-panel /) }

@default { (app-user-panel /) }

}

```

track in @for -- Why It Matters

track product.id tells Angular which property uniquely identifies each item. Without it Angular re-renders the entire list on every change. With it, Angular only updates what actually changed. Always track by a unique id.

F4. Component Lifecycle Hooks

Think of a component's life like a person's day:

- * Born -) constructor (don't do much here)
- * Briefed -) ngOnChanges (inputs arrived)
- * Starts work -) ngOnInit (data loading, subscriptions)
- * Checks the clock -) Angular's change detection runs
- * Sees the room -) ngAfterViewInit (DOM is ready)
- * Goes home -) ngOnDestroy (cleanup)

```

@Component({ selector: 'app-user', template: '...' })

export class UserComponent implements OnInit, OnChanges,
AfterViewInit, OnDestroy {

@Input() userId!: number; // arrives before ngOnInit

// Runs when @Input() values change

// Runs BEFORE ngOnInit on first load

ngOnChanges(changes: SimpleChanges): void {

if (changes['userId']) {

console.log('userId changed:', changes['userId'].currentValue);

}
}

```

```

}

// Runs ONCE after first ngOnChanges

// Best place for: HTTP calls, subscriptions, setup

ngOnInit(): void {

  this.loadUser(this.userId);

}

// Runs after the template and child components are rendered

// Best place for: @ViewChild access, DOM manipulation

ngAfterViewInit(): void {

  this.inputRef?.nativeElement.focus();

}

// Runs just before component is destroyed

// Best place for: unsubscribe, cleanup, cancel timers

ngOnDestroy(): void {

  this.subscription.unsubscribe();

}

}

```

The 80% Rule

Your study plan is right -- ngOnInit and ngOnDestroy cover 80% of all lifecycle needs:

- * ngOnInit = start the work (load data, set up subscriptions)
- * ngOnDestroy = clean up (unsubscribe, cancel timers)
- * ngOnChanges = when you need to react to @Input() changes
- * ngAfterViewInit = when you need to access the DOM

F5. @Input() and @Output() -- Component Communication

The One-Way Street Rule

Angular data flows down through @Input() and events bubble up through @Output(). Like a chain of command -- instructions go down, reports come up.

```

Parent Component

[user]="selectedUser" @Input() -- data goes DOWN

(userSelected)="..." @Output() -- events go UP

```

Child Component

```
// ---- CHILD COMPONENT ----

@Component({
  selector: 'app-user-card',
  standalone: true,
  template: `
    (div class="card")
    (h3){ { user.name } }(/h3)
    (button (click)="onSelect()")Select(/button)
    (button (click)="onDelete()")Delete(/button)
  (/div)
  `
})

export class UserCardComponent {

  // @Input() -- receives data FROM parent
  @Input({ required: true }) user!: User; // required in Angular 16+
  @Input() isSelected = false;

  // @Output() -- sends events TO parent
  @Output() userSelected = new EventEmitter<User>();
  @Output() userDeleted = new EventEmitter<number>(); // emits the id

  onSelect(): void { this.userSelected.emit(this.user); }
  onDelete(): void { this.userDeleted.emit(this.user.id); }
}

// ---- PARENT COMPONENT ----

@Component({
  selector: 'app-user-list',
  standalone: true,
```

```

imports: [UserCardComponent, CommonModule],

template: `

@for (user of users; track user.id) {

  (app-user-card

    [user]="user"

    [isSelected]="selectedId === user.id"

    (userSelected)="onUserSelected($event)"

    (userDeleted)="onUserDeleted($event)"

  (/app-user-card)

}

`

}))

export class UserListComponent {

  users: User[] = [];

  selectedId: number | null = null;

  onUserSelected(user: User): void {

    this.selectedId = user.id;

  }

  onUserDeleted(id: number): void {

    // Immutable update -- Topic 7 [Y]

    this.users = this.users.filter(u => u.id !== id);

  }

}

```

New Angular 17+ Syntax -- input() and output() functions

Angular is moving away from decorators toward functions for inputs and outputs:

```

import { input, output } from '@angular/core';

@Component({ ... })

export class UserCardComponent {

```



```

// input() function -- replaces @Input() decorator

user = input.required(User()); // required

isSelected = input(false); // with default value


// output() function -- replaces @Output() + EventEmitter

userSelected = output(User());

userDeleted = output(number());


onSelect(): void { this.userSelected.emit(this.user()); }

// Note: input() returns a signal -- access with ()

}

```

Foundation Summary -- The Mental Model

Angular App

main.ts -) bootstrapApplication(AppComponent, appConfig)

app.config.ts -) provideRouter(), provideHttpClient()

AppComponent (root -- always visible)

template uses {{ }}, [], (), [()]

@if / @for / @switch for logic

ChildComponent A

receives data via @Input()

sends events via @Output()

ChildComponent B

same pattern

Services (shared logic, injected via constructor)

@Injectable({ providedIn: 'root' })

Four things every Angular component does:

1. Receives data -- via @Input() from parent
2. Displays data -- via template ({{ }}, @if, @for)

3. Reacts to events -- via (click), (input) etc.
4. Communicates up -- via @Output() to parent

Now that the foundations are clear, here is Topic 20 in full using the master prompt format.

Topic 20 -- RxJS Subscription Management

0. Difficulty and Priority Rating

- * Difficulty: Intermediate -- creating subscriptions is easy; understanding why unmanaged ones cause memory leaks and knowing the modern patterns to prevent them takes real practice
 - * Angular Relevance: Critical -- every HTTP call, every real-time data stream, every form value change in Angular produces an Observable that needs to be subscribed to and cleaned up; one unmanaged subscription in a navigated-away component silently keeps it alive in memory forever
-

1. Explanation

The Newspaper Subscription Analogy

Imagine you subscribe to a newspaper delivery. Every morning a newspaper arrives at your door. You move house -- but you forget to cancel the subscription. The newspaper delivery continues arriving at your old address forever. The delivery van keeps going there. The printing press keeps printing your copy. Resources are consumed indefinitely for a house nobody lives in.

An unmanaged Angular subscription is that forgotten newspaper delivery.

You navigate away from a component -- Angular destroys it. But if its Observable subscriptions weren't cancelled, they keep running. HTTP responses might try to update a component that no longer exists. Event streams keep firing callbacks into the void. Memory that should be freed stays occupied.

```
Subscribe = start the newspaper delivery  
  
Unsubscribe = cancel when you move house  
  
Component destroyed without unsubscribe  
  
= delivery to an empty house -- forever
```

The Tap That Won't Turn Off Analogy

Picture a tap left running after you've finished washing up. The water keeps flowing -- down the drain, wasted. The longer you leave it, the more is wasted. In a large app with dozens of components subscribing and never unsubscribing, it's dozens of taps all running simultaneously.

Memory leaks are taps that weren't turned off.

Each unmanaged subscription holds a reference to its component, preventing the garbage collector from freeing that memory. Over time, navigating back and forth in an app creates more and more ghost components -- all still receiving data, all taking up memory, all potentially causing unexpected behaviour.

In Plain English

- * Subscription = the active connection between an Observable and its observer -- like the newspaper delivery being active

- * Unsubscribe = cancelling the delivery -- the Observable stops sending, memory is freed
- * Memory leak = a component is "destroyed" but its subscriptions keep it alive in memory -- the tap left running
- * takeUntilDestroyed = a modern Angular operator that automatically cancels subscriptions when the component is destroyed -- the smart tap that turns itself off
- * async pipe = lets Angular manage the subscription for you in the template -- Angular subscribes when the component creates, unsubscribes when it destroys

Now the Technical Terms

- * Subscription = the object returned by .subscribe() -- calling .unsubscribe() on it cancels the Observable
- * ngOnDestroy = the Angular lifecycle hook where manual cleanup happens
- * takeUntilDestroyed() = an RxJS operator introduced in Angular 16 that completes the Observable when the component's DestroyRef fires
- * DestroyRef = Angular's injectable service that notifies when the current context (component/service) is destroyed
- * async pipe = Angular's built-in template pipe that subscribes to an Observable and automatically unsubscribes on component destruction

2. Connection to Prior Concepts

Subscription management is where everything from the first 19 topics converges:

- * Topic 3 (Closures) -- every .subscribe() callback is a closure over the component instance via this. If the subscription isn't cleaned up, the closure keeps the component's entire scope alive in memory -- the tent that never came down
- * Topic 5 (Event Loop) -- subscriptions receive data from the queue asynchronously. An unsubscribed stream keeps putting callbacks on the queue even after the component is gone
- * Topic 6 (Callbacks) -- .subscribe({ next, error, complete }) -- these three callbacks are what you're managing. Unsubscribing stops them from being called
- * Topic 18 (Error Handling) -- the error callback in subscribe is your last line of defence for RxJS errors. Topic 18's catchError protects the stream; the error handler here catches anything that slipped through
- * Topic 19 (?. and ??) -- unmanaged subscriptions often cause "cannot set property of null" errors -- the component is destroyed but the callback fires and tries to set this.data. ?. on this doesn't help here -- unsubscribing is the real fix

Bridging note for your records:

"Every .subscribe() you write creates a tap. The three modern ways to turn off the tap: takeUntilDestroyed (automatic, recommended), async pipe (template-level, cleanest), or manual unsubscribe() in ngOnDestroy (always works, always valid). The goal is simple: if the component is gone, the subscription must be gone too."

3. Code Example

Example 1 -- The problem -- what happens without cleanup

Showing exactly what a memory leak looks like and why it's dangerous.

```
// [N] THE LEAK -- subscription never cancelled
```

```

@Component({ selector: 'app-user-list', template: '...' })

export class UserListComponent implements OnInit {

  users: User[] = [];

  constructor(private readonly userService: UserService) {}

  ngOnInit(): void {
    // Creates a subscription -- but never stores or cancels it
    this.userService.getUsers().subscribe(users => {
      this.users = users; // updates component state
    });

    // If this is a polling Observable (updates every 5 seconds):
    interval(5000).pipe(
      switchMap(() => this.userService.getUsers())
    ).subscribe(users => {
      this.users = users; // fires every 5 seconds -- forever
    });
  }

  // No ngOnDestroy -- no unsubscribe
  // User navigates away -- component "destroyed"
  // But subscriptions keep firing -- every 5 seconds -- forever
  // 'this.users = users' runs on a destroyed component
  // ERROR: Cannot set properties of null (reading 'users') [N]
  // Memory: component stays in memory -- GC can't collect it [N]
}

```

Example 2 -- Manual unsubscribe -- the traditional pattern

Always works -- the baseline you should always know.

```

// [Y] MANUAL UNSUBSCRIBE -- traditional pattern

import { Subscription } from 'rxjs';

@Component({ selector: 'app-user-list', template: '...' })

```

```

export class UserListComponent implements OnInit, OnDestroy {

  users: User[] = [];

  // Store the subscription so we can cancel it later
  private readonly sub = new Subscription();

  constructor(private readonly userService: UserService) {}

  ngOnInit(): void {
    // .add() collects multiple subscriptions into one
    this.sub.add(
      this.userService.getUsers().subscribe(users => {
        this.users = users;
      })
    );

    // Add as many as you need -- all cancelled together
    this.sub.add(
      this.userService.getNotifications().subscribe(notif => {
        this.notifications = notif;
      })
    );
  }

  ngOnDestroy(): void {
    // One call cancels ALL subscriptions added via .add()
    this.sub.unsubscribe(); // [Y] taps turned off -- memory freed
  }
}

// ---- Alternative -- array of subscriptions ----

export class UserListComponent implements OnInit, OnDestroy {

```

```

private readonly subs: Subscription[] = [];

ngOnInit(): void {
  this.subs.push(
    this.userService.getUsers().subscribe(users => {
      this.users = users;
    })
  );
}

ngOnDestroy(): void {
  this.subs.forEach(s => s.unsubscribe()); // cancel all [Y]
}
}

```

Example 3 -- takeUntil -- the classic RxJS pattern

Before takeUntilDestroyed existed -- still widely used in existing codebases.

```

// [Y] takeUntil -- the pre-Angular-16 pattern

// Still valid -- you'll see this everywhere in existing Angular codebases

import { Subject, takeUntil } from 'rxjs';

@Component({ selector: 'app-user-list', template: '...' })
export class UserListComponent implements OnInit, OnDestroy {

  users: User[] = [];

  // A Subject that fires once -- when the component is destroyed
  // Think of it as a "shutdown signal"
  private readonly destroy$ = new Subject<void>();

  constructor(private readonly userService: UserService) {}

  ngOnInit(): void {
    this.userService.getUsers().pipe(

```

```

takeUntil(this.destroy$) // "keep going UNTIL destroy$ fires"

).subscribe(users => {

this.users = users;

});

// Every subscription uses the same destroy$ signal
interval(5000).pipe(
switchMap(() => this.userService getUsers()),
takeUntil(this.destroy$) // same pattern -- all cancelled together
).subscribe(users => {

this.users = users;

});
}

ngOnDestroy(): void {

this.destroy$.next(); // fire the shutdown signal

this.destroy$.complete(); // clean up the Subject itself

// All takeUntil(this.destroy$) subscriptions cancel automatically [Y]

}

}

```

Angular Example -- takeUntilDestroyed -- the modern pattern

The Angular 16+ way -- cleanest, least boilerplate, recommended going forward.

```

// [Y] takeUntilDestroyed -- Angular 16+ modern pattern

// No Subject needed, no ngOnDestroy needed for subscription cleanup

import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
import { DestroyRef, inject } from '@angular/core';

@Component({

selector: 'app-order-list',

standalone: true,

imports: [CommonModule],

template: `

```

```

    (div *ngIf="isLoading")Loading orders...(/div)

    (div *ngFor="let order of orders; trackBy: trackById")

    {{ order.id }} -- {{ order.status }}

    (/div)
  `
})

export class OrderListComponent implements OnInit {

  orders: Order[] = [];

  isLoading = false;

  // inject() works outside constructor in Angular 14+
  private readonly destroyRef = inject(DestroyRef);

  constructor(
    private readonly orderService: OrderService,
    private readonly userService: UserService
  ) {}

  ngOnInit(): void {
    this.isLoading = true;

    // takeUntilDestroyed -- automatically cancels when component is destroyed
    // No ngOnDestroy needed for this [Y]
    this.orderService.getOrders().pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe({
      next: orders => { this.orders = orders; this.isLoading = false; },
      error: () => { this.isLoading = false; },
      complete: () => { this.isLoading = false; }
    });

    // Multiple subscriptions -- same destroyRef -- all auto-cancelled
    this.userService.getCurrentUser().pipe(

```



```

takeUntilDestroyed(this.destroyRef)

).subscribe(user => {

this.currentUser = user;

});

// Works with polling, WebSockets, form valueChanges -- anything
interval(30_000).pipe(
switchMap(() => this.orderService.getOrders()),
takeUntilDestroyed(this.destroyRef) // auto-cancels the interval too [Y]
).subscribe(orders => {

this.orders = orders;

});
}

trackById = (index: number, order: Order) => order.id;

// ngOnDestroy not needed for subscription cleanup [Y]
// Angular handles it via DestroyRef
}

// ---- EVEN CLEANER -- takeUntilDestroyed without explicit destroyRef ----
// When called inside an injection context (constructor, field initializer)
// destroyRef is inferred automatically

@Component({ selector: 'app-dashboard', template: '...' })
export class DashboardComponent {

// [Y] Subscription set up in field initializer -- injection context
// takeUntilDestroyed() with no argument -- infers destroyRef automatically
private readonly data$ = this.dataService.getDashboardData().pipe(
takeUntilDestroyed() // no argument needed here [Y]
);

```

```
constructor(private readonly dataService: DataService) {}  
  
}
```

4. Before and After Refactor

The Problem in Plain English:

Imagine hiring contractors for every renovation project in your house but never telling them when the job is done. Each one just keeps working indefinitely -- hammering, drilling, using electricity -- long after you've sold the house and moved out. The new owners are confused. Resources are being drained. Nobody told the contractors to stop.

Every unmanaged subscription is a contractor who was never told the job is done. The `takeUntilDestroyed` pattern is the project manager who automatically dismisses all contractors the moment the renovation is complete.

```
// [N] BEFORE -- subscriptions scattered, manually managed, easy to miss  
  
@Component({ selector: 'app-profile', template: '...' })  
export class ProfileComponent implements OnInit, OnDestroy {  
  
  user: User | null = null;  
  
  notifications: Notification[] = [];  
  
  stats: UserStats | null = null;  
  
  isLoading = false;  
  
  // Three separate subscription variables -- easy to miss one  
  
  private userSub: Subscription;  
  
  private notifSub: Subscription;  
  
  private statsSub: Subscription;  
  
  constructor(  
  
    private userService: UserService,  
  
    private notifService: NotificationService,  
  
    private statsService: StatsService  
  ) {}  
  
  ngOnInit(): void {  
  
    this.isLoading = true;
```

```

this.userSub = this.userService.getCurrentUser().subscribe(user => {
  this.user = user;
});

this.notifSub = this.notifService.getNotifications().subscribe(notif => {
  this.notifications = notif;
});

this.statsSub = this.statsService.getUserStats().subscribe(stats => {
  this.stats = stats;
  this.isLoading = false;
});

// Fourth subscription added later -- developer forgot to add cleanup
interval(60_000).pipe(
  switchMap(() => this.statsService.getUserStats())
).subscribe(stats => {
  this.stats = stats; // [N] NO cleanup -- memory leak
});
}

ngOnDestroy(): void {
  // Three unsubscribes -- but the interval is still running [N]
  this.userSub?.unsubscribe();
  this.notifSub?.unsubscribe();
  this.statsSub?.unsubscribe();

  // Interval subscription forgotten -- tap still running [N]
}
}

// [Y] AFTER -- takeUntilDestroyed -- automatic, impossible to miss

@Component({ selector: 'app-profile', template: '...' })

```

```

export class ProfileComponent implements OnInit {

  user: User | null = null;

  notifications: Notification[] = [];

  stats: UserStats | null = null;

  isLoading = false;

  // No subscription variables -- no manual cleanup needed
  private readonly destroyRef = inject(DestroyRef);

  constructor(
    private readonly userService: UserService,
    private readonly notifService: NotificationService,
    private readonly statsService: StatsService
  ) {}

  ngOnInit(): void {
    this.isLoading = true;

    // Every subscription uses the same pattern -- impossible to forget [Y]
    this.userService.getCurrentUser().pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(user => { this.user = user; });

    this.notifService.getNotifications().pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(notif => { this.notifications = notif; });

    this.statsService.getUserStats().pipe(
      takeUntilDestroyed(this.destroyRef)
    ).subscribe(stats => { this.stats = stats; this.isLoading = false; });

    // New subscription added later -- same pattern -- automatically cleaned [Y]
    interval(60_000).pipe(

```

```

switchMap(() => this.statsService.getUserStats()),

takeUntilDestroyed(this.destroyRef) // [Y] auto-cancelled on destroy

).subscribe(stats => { this.stats = stats; });

}

// No ngOnDestroy needed -- Angular handles all cleanup via destroyRef [Y]

}

```

Why it matters: The before version has four subscriptions and only three unsubscribes -- one memory leak hiding in plain sight. In a real app with dozens of components, these accumulate silently until performance degrades and errors like "cannot set property of destroyed component" start appearing. The after version makes forgetting impossible -- the pattern is identical for every subscription and Angular handles the cleanup automatically.

5. Common Mistakes and Misconceptions

"HTTP calls don't need unsubscribing -- they complete automatically"

This is partially true but dangerously incomplete. A single HTTP call with `HttpClient` does complete after one response -- so technically one-shot subscriptions don't leak. But this reasoning leads developers to never unsubscribe anything, because "it probably completes." Then a polling `Observable`, a `WebSocket`, a form `valueChanges` stream, or a `BehaviorSubject` causes a massive leak -- and the developer doesn't understand why.

Think of it like this: most newspaper deliveries do eventually run out of editions -- but some run indefinitely. If you make a habit of never cancelling any subscription, the infinite ones will quietly destroy your app's memory while you wonder why.

```

// [Y] TECHNICALLY safe -- HttpClient completes after one response

this.http.get('/api/users').subscribe(users => {

  this.users = users;

});

// This won't leak -- BUT...

// [N] These absolutely will leak without unsubscribe:

this.searchControl.valueChanges.subscribe(...); // infinite -- never completes

interval(5000).subscribe(...); // infinite -- never completes

this.store.select(getUsers).subscribe(...); // infinite -- never completes

this.websocket.messages$.subscribe(...); // infinite -- never completes

// [Y] Best practice -- use takeUntilDestroyed for ALL subscriptions

// Don't try to reason about which ones complete -- just always clean up

```

```

this.http.get('/api/users').pipe(
  takeUntilDestroyed(this.destroyRef)
).subscribe(users => { this.users = users; });

// Slightly redundant for HTTP -- but consistent and safe [Y]

```

"Subscribing multiple times to the same Observable is fine -- it's the same stream"

Every `.subscribe()` call creates a new, independent subscription -- even to the same Observable. If you accidentally subscribe twice (e.g. by calling a method that subscribes inside `ngOnChanges` without cleaning up the previous one), you get duplicate HTTP requests, duplicate state updates, and duplicate callbacks running.

Think of it like accidentally signing up for the same newspaper twice -- you get two copies every morning. Call a subscription method three times and you get three copies. Each navigation cycle adds more.

```

// [N] Subscribing multiple times -- each ngOnChanges call adds a new one

@Component({ ... })

export class UserComponent implements OnChanges {

  @Input() userId: number;

  ngOnChanges(): void {

    // Every time userId changes -- a NEW subscription is created

    // Previous one is NOT cancelled

    // After 5 userId changes: 5 active subscriptions [N]

    this.userService.getUser(this.userId).subscribe(user => {

      this.user = user;

    });

  }

}

// [Y] Use takeUntilDestroyed OR switchMap to handle changing @Inputs

@Component({ ... })

export class UserComponent implements OnInit {

  @Input() userId: number;

  user: User | null = null;

```

```

private readonly destroyRef = inject(DestroyRef);

// Use a Subject to trigger new loads when userId changes
private readonly userIdChange$ = new Subject<number>();

ngOnInit(): void {
  this.userIdChange$.pipe(
    switchMap(id => this.userService.getUser(id)), // cancels previous
    takeUntilDestroyed(this.destroyRef)
  ).subscribe(user => { this.user = user; });
}

ngOnChanges(changes: SimpleChanges): void {
  if (changes['userId']) {
    this.userIdChange$.next(this.userId); // emit new id -- switchMap handles it
  }
}
}

```

"Using async pipe and manual subscribe together is fine -- more control"

Mixing async pipe and manual `.subscribe()` on the same Observable is a classic double-subscription bug. The async pipe creates one subscription. Your manual `.subscribe()` creates another. You get duplicate HTTP calls, duplicate state updates, and potentially duplicate side effects.

```

// [N] Double subscription -- async pipe AND manual subscribe

@Component({
  template: `
    (!-- async pipe creates subscription #1 --)
    (div *ngFor="let user of users$ | async" ){{ user.name }}(/div)
  `
})
export class UserComponent implements OnInit {

  users$ = this.userService.getUsers(); // Observable

  ngOnInit(): void {

```

```

// Manual subscribe creates subscription #2 -- same Observable

this.users$.subscribe(users => {

  console.log('loaded', users.length); // fires twice [N]

  // HTTP call made TWICE [N]

});

}

}

// [Y] Pick one pattern -- async pipe OR manual subscribe -- never both
// Option A: async pipe only (cleanest for templates)

@Component({

  template: `(div *ngFor="let user of users$ | async"){{ user.name }}(/div)`

})

export class UserComponent {

  readonly users$ = this.userService getUsers();

  // No subscribe needed -- async pipe handles everything [Y]

}

// Option B: manual subscribe only (needed for side effects)

@Component({

  template: `(div *ngFor="let user of users"){{ user.name }}(/div)`

})

export class UserComponent implements OnInit {

  users: User[] = [];

  private readonly destroyRef = inject(DestroyRef);

  ngOnInit(): void {

    this.userService getUsers().pipe(

      takeUntilDestroyed(this.destroyRef)

    ).subscribe(users => {

      this.users = users;

      this.analytics.track('users_loaded', users.length); // side effect

```



```
});  
  
}  
  
}
```

6. Do's and Don'ts

| [Y] Do | [N] Don't |

|---|---|

| Use `takeUntilDestroyed(this.destroyRef)` for all subscriptions in Angular 16+ | Leave subscriptions without any cleanup -- even "one-shot" HTTP subscriptions |

| Use `async` pipe for template-bound Observables -- Angular manages the lifecycle | Mix `async` pipe AND manual `.subscribe()` on the same Observable -- double subscription |

| Collect multiple subscriptions with `new Subscription()` and `.add()` when using manual `unsubscribe` | Create individual Subscription variables for each stream -- easy to miss one in `ngOnDestroy` |

| Use `switchMap` for subscriptions triggered by `@Input()` changes -- cancels previous call | Re-subscribe on every `ngOnChanges` without cancelling the previous subscription |

| Use `takeUntil(this.destroy$)` when working in Angular (16 codebases | Use `takeUntilDestroyed` in Angular (16 -- it doesn't exist |

| Always handle error and complete in `.subscribe()` -- especially complete when using `EMPTY` as error fallback | Only handle next in subscribe -- errors and completes need attention too |

7. Debugging Tips

Bug: ERROR: Cannot set properties of null (reading 'users')

Cannot set properties of null -- after navigating away from component

* Plain English cause: Subscription callback fired after component was destroyed -- this is null -- the tap was left running

* Fix: Add `takeUntilDestroyed(this.destroyRef)` to every subscription

* Angular note: This is the #1 symptom of a memory leak in Angular -- the error appears in the console after navigation

Bug: HTTP request firing multiple times unexpectedly

Network tab shows same endpoint called 2, 3, or 4 times

* Plain English cause: Multiple subscriptions to the same Observable -- either double-subscribing manually, or `async` pipe + manual subscribe, or subscribing in `ngOnChanges` without cancelling the previous

* Fix: Audit all `.subscribe()` calls -- ensure each Observable is subscribed to exactly once

Bug: Real-time data still updating after navigating away

Console logs firing, state updating, even on a different route

* Plain English cause: `interval`, `WebSocket`, or `BehaviorSubject` subscription never unsubscribed -- ghost component still alive

* Fix: Verify `takeUntilDestroyed` is on every long-lived subscription

Bug: takeUntilDestroyed() with no argument throws error

Error: takeUntilDestroyed() can only be used within an injection context

* Plain English cause: takeUntilDestroyed() without an argument must be called inside an injection context (constructor, field initialiser, inject() call). If called inside ngOnInit, it can't auto-detect the DestroyRef

* Fix: Either pass this.destroyRef explicitly, or move the subscription setup to a field initialiser

```
// [N] No injection context -- ngOnInit is not an injection context

ngOnInit(): void {

  this.data$.pipe(

    takeUntilDestroyed() // [N] Error -- no destroyRef available here

  ).subscribe();

}

// [Y] Option 1 -- pass destroyRef explicitly

private readonly destroyRef = inject(DestroyRef);

ngOnInit(): void {

  this.data$.pipe(

    takeUntilDestroyed(this.destroyRef) // [Y] explicit -- always works

  ).subscribe();

}

// [Y] Option 2 -- field initialiser (injection context)

private readonly data$ = this.dataService.getData().pipe(

  takeUntilDestroyed() // [Y] field initialiser IS an injection context

);
```

8. Real-World Applications

Application 1 -- Real-time Dashboard with Multiple Streams

Plain English first:

A dashboard typically has several live data streams -- user stats, notifications, activity feed, system status. Each needs to update in real time. Without proper subscription management, navigating away from the dashboard and back creates ghost instances -- each navigation cycle adding more active streams, each consuming more memory, until the app slows to a crawl. With takeUntilDestroyed, every stream is automatically cancelled the moment the dashboard is left, and fresh streams are created on return.

```

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [CommonModule],
  template: `
    (div class="stats-grid")
    (div)Active Users: {{ stats?.activeUsers ?? 0 }}(/div)
    (div)Orders Today: {{ stats?.ordersToday ?? 0 }}(/div)
    (div)Revenue: {{ stats?.revenue?.toFixed(2) ?? '0.00' }}(/div)
  (/div)

  (div class="notifications")
  (div *ngFor="let notif of notifications; trackBy: trackById")
  {{ notif.message }}
  (/div)
  (/div)
  `
})

export class DashboardComponent implements OnInit {

  stats: DashboardStats | null = null;
  notifications: Notification[] = [];
  isLoading = false;

  private readonly destroyRef = inject(DestroyRef);

  constructor(
    private readonly statsService: StatsService,
    private readonly notifService: NotificationService
  ) {}

  ngOnInit(): void {
    // Polling stream -- updates every 30 seconds

```

```

// takeUntilDestroyed cancels the interval AND all HTTP calls [Y]

timer(0, 30_000).pipe(

switchMap(() => this.statsService.getDashboardStats()),

takeUntilDestroyed(this.destroyRef)

).subscribe(stats => { this.stats = stats; });

// WebSocket or SSE stream -- fires on every new notification

this.notifService.notifications$.pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(notif => {

this.notifications = [notif, ...this.notifications].slice(0, 10);

});

}

trackById = (i: number, n: Notification) => n.id;

}

```

Application 2 -- Search with Live Results

Plain English first:

A search box that fetches results as the user types is a classic subscription management challenge. Without `switchMap` and `takeUntilDestroyed`, every keystroke creates a new HTTP request that never cancels the previous ones. Results arrive out of order, the component leaks memory, and the wrong results get displayed. With proper subscription management, each new keystroke cancels the previous request, results always match the current query, and everything is cleaned up on navigation.

```

@Component({

selector: 'app-product-search',

standalone: true,

imports: [CommonModule, ReactiveFormsModule],

template: `

(input [formControl]="searchControl" placeholder="Search products...")



Searching...(</div>



Angular Learning Series -- Angular Foundations



Page 28


```

```

(p *ngIf="!isSearching andand results.length === 0 andand searchControl.value")
No results for "{{ searchControl.value }}"

(/p)
`

}))

export class ProductSearchComponent implements OnInit {

  readonly searchControl = new FormControl('');

  results: Product[] = [];

  isSearching = false;

  private readonly destroyRef = inject(DestroyRef);

  constructor(private readonly productService: ProductService) {}

  ngOnInit(): void {
    this.searchControl.valueChanges.pipe(
      debounceTime(300), // wait 300ms after last keypress
      distinctUntilChanged(), // only if value actually changed
      filter(q => (q?.length ?? 0) >= 2), // min 2 characters
      tap(() => { this.isSearching = true; this.results = []; }),
      switchMap(query => // cancel previous -- start new
        this.productService.search(query ?? '').pipe(
          catchError(() => of([])) // error inside switchMap -- Topic 18
        )
      ),
      takeUntilDestroyed(this.destroyRef) // clean up on component destroy
    ).subscribe(results => {
      this.results = results;

      this.isSearching = false;
    });
  }
}

```

```
trackById = (i: number, p: Product) => p.id;

}
```

9. Practice Exercises

Exercise 1 -- Beginner

Build an `ActivityFeedComponent` that polls an `ActivityService.getFeed()` Observable every 10 seconds and displays the last 5 activities. First implement it WITHOUT any cleanup -- observe the memory leak by navigating away and back (add a `console.log` in the callback to see it firing on a "destroyed" component). Then add `takeUntilDestroyed` -- verify the logs stop when you navigate away. Document what you observe in both cases.

Exercise 2 -- Intermediate

Build a `UserSearchComponent` with a text input. As the user types, search a `UserService.search(query)` Observable with at least 300ms debounce. The component should display: search results, a loading indicator, an error message if the search fails, and a "no results" message for empty results. Use `takeUntilDestroyed` throughout. Add a second subscription for a `UserService.getRecentSearches()` stream that updates the "recent searches" list in real time. Verify that navigating away and back creates exactly one active subscription each time -- not accumulating.

Exercise 3 -- Advanced

Build a `DataSyncComponent` that manages three simultaneous streams: user profile (updates every 60 seconds), unread notification count (`WebSocket` -- simulate with `interval(5000)`), and a search stream triggered by an `@Input()` `searchQuery`. When `searchQuery` changes, cancel the previous search and start a new one -- handle rapid changes gracefully with `switchMap`. All three streams must be managed with `takeUntilDestroyed`. Add a `connectionStatus` property that shows "connected" when all streams are active and "reconnecting" when any stream errors and retries. Verify zero memory leaks by tracking active subscription count in a service.

10. Key Takeaways

Core Concepts in Plain English

- * Subscription = active connection between Observable and observer -- like a newspaper delivery being active
- * Memory leak = destroyed component whose subscriptions are still running -- the tap left on
- * `takeUntilDestroyed` = Angular 16+ automatic cleanup -- the smart tap that turns itself off when you leave
- * async pipe = template-level subscription management -- Angular subscribes and unsubscribes automatically
- * Manual unsubscribe = always valid -- `Subscription.add()` + `ngOnDestroy` -- the baseline
- * `switchMap` = cancels the previous inner Observable when a new value arrives -- essential for search, `@Input()` changes

The Three Modern Patterns

```
Pattern 1 -- takeUntilDestroyed (recommended for most cases)

private readonly destroyRef = inject(DestroyRef);
```

```
observable$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe(...)
```

Pattern 2 -- async pipe (cleanest for templates)

```
readonly data$ = this.service.getData();
```

```
template: {{ data$ | async }}
```

Pattern 3 -- manual unsubscribe (always works, pre-Angular 16)

```
private readonly sub = new Subscription();
```

```
ngOnInit(): void { this.sub.add(observable$.subscribe(...)); }
```

```
ngOnDestroy(): void { this.sub.unsubscribe(); }
```

Quick Reference Table

| Scenario | Pattern | Why |

|---|---|---|

| Template data display | async pipe | Angular manages lifecycle, no subscribe needed |

| Side effects needed | takeUntilDestroyed | Full control, auto cleanup |

| Angular (16 | takeUntil(destroy\$) or manual | takeUntilDestroyed not available |

| @Input() triggers new stream | switchMap + takeUntilDestroyed | Cancels previous stream on new input |

| Polling / intervals | timer(0, ms) + switchMap + takeUntilDestroyed | All cancelled together on destroy |

Angular-Specific Rules

- * takeUntilDestroyed() with no argument = must be in injection context (field initialiser, constructor)
- * takeUntilDestroyed(this.destroyRef) with explicit ref = works anywhere including ngOnInit
- * Never mix async pipe AND manual .subscribe() on the same Observable -- double subscription
- * Use switchMap for streams triggered by @Input() changes -- cancels previous automatically
- * One-shot HTTP calls technically don't leak -- but use takeUntilDestroyed anyway for consistency

One-Line Memory Aid

) Every .subscribe() is a tap you turned on. takeUntilDestroyed is the smart valve that turns it off when you leave the room. async pipe turns it on and off for you. Never leave a tap running in an empty house.

11. Thought-Provoking Question + Answer

) Angular 17+ Signals are being positioned as a replacement for much of RxJS in Angular apps. If Signals manage state reactively without subscriptions -- does subscription management become irrelevant? Or will RxJS subscriptions always be needed, and why?

Plain English explanation first:

Think of it like the invention of automatic doors. Before, every building needed someone to physically open and close doors -- manual effort, easy to forget. Automatic doors handle themselves. But automatic doors don't replace all doors -- there are still places where manual doors make sense,

where automatic ones can't work, or where the plumbing for automatic doors isn't installed yet.

Signals are automatic doors for state -- reactive, self-managing, no subscription needed. RxJS Observables are powerful manual doors -- more flexible, handling complex async flows, integrations, and time-based operations that signals can't do yet.

```
// SIGNALS -- no subscription needed

// Perfect for: local component state, derived state, simple shared state

@Component({ template: `(p)Count: {{ count() }}(/p)` })
export class CounterComponent {

  count = signal(0); // no subscribe [Y]

  doubled = computed(() => this.count() * 2); // auto-derived [Y]

  increment(): void { this.count.update(c => c + 1); } // no leak possible [Y]
}


// RxJS OBSERVABLES -- subscriptions still needed

// Perfect for: HTTP calls, WebSockets, complex async chains,
// time-based operations, external event streams

@Component({ ... })
export class SearchComponent implements OnInit {
  private readonly destroyRef = inject(DestroyRef);

  ngOnInit(): void {
    // RxJS handles what signals can't:

    this.searchControl.valueChanges.pipe( // continuous event stream
      debounceTime(300), // time-based operator
      switchMap(q => this.http.get(`/search?q=${q}`)), // HTTP + cancellation
      takeUntilDestroyed(this.destroyRef) // still needed [Y]
    ).subscribe(results => this.results.set(results)); // set a signal from RxJS [Y]
  }
}
```

The current reality -- signals and RxJS coexist:

Signals replace RxJS for:

[Y] Component local state (count, isLoading, selectedItem)

[Y] Derived/computed values (filteredList, totalPrice)

[Y] Simple shared state between components

[Y] Replacing BehaviorSubject in services for state

RxJS is still needed for:

[Y] HTTP calls (HttpClient returns Observable -- not changing)

[Y] WebSockets and SSE (real-time streams)

[Y] Complex operator chains (debounce, switchMap, retry)

[Y] Form valueChanges (Angular forms still use Observables)

[Y] Third-party libraries (most return Observables)

[Y] RxJS-specific timing (timer, interval, delay)

The bridge pattern -- RxJS data into Signals:

```
readonly users = toSignal(  
  this.userService.getUsers(), // Observable source  
  { initialValue: [] } // signal's initial value  
);  
  
// Observable is subscribed internally by toSignal()  
// Subscription managed by Angular automatically [Y]  
  
// Template accesses users() like a signal -- no async pipe needed [Y]
```

Practical rule to take away:

) "Signals dramatically reduce how much RxJS you write for state management -- but they don't eliminate subscription management. HTTP calls, WebSockets, form streams, and complex async chains are still Observable-based and still need takeUntilDestroyed. The emerging best practice is: use signals for state, use RxJS for async operations, and use toSignal() to bridge the two -- which manages the subscription for you automatically. Master both -- they're complementary, not competing."

12. Related Topics to Learn Next

Angular-Specific Concepts

takeUntilDestroyed and DestroyRef (Topic 21 -- deep dive into the mechanism powering this topic)*

async pipe (Topic 22 -- the template alternative to manual subscription management)*

Angular Signals (Topic 23 -- the technology reducing RxJS needs -- toSignal() bridges both)*

OnPush Change Detection (Topic 24 -- subscriptions + signals + OnPush = the performance trinity)*

RxJS Operators in depth (Topic 46 -- switchMap, mergeMap, forkJoin -- the operators that make subscription management complex)*

Updated Learning Tracker

[Y] Topics 1-19 -- ES6/JS + TypeScript Foundation COMPLETE

[Y] F1-F5 -- Angular Foundations Overview COMPLETE

[Y] Topic 20 -- RxJS Subscription Management COMPLETE

-) Topic 21 -- takeUntilDestroyed and DestroyRef NEXT

Key Concept Added to Quick Reference

Topic 20 -- RxJS Subscription Management

) Every `.subscribe()` is a tap you turned on. `takeUntilDestroyed` turns it off when you leave. `async pipe` does it automatically. Never leave a tap running in an empty house.

- * Subscription = active connection -- must be cancelled when component is destroyed
- * Memory leak = destroyed component with active subscriptions -- ghost component in memory
- * `takeUntilDestroyed(destroyRef)` = Angular 16+ -- automatic cleanup -- recommended default
- * `async pipe` = template subscription -- Angular manages entire lifecycle
- * Manual unsubscribe = `Subscription.add()` + `ngOnDestroy` -- always valid, pre-16 pattern
- * `switchMap` = cancels previous inner stream -- essential for search and `@Input()` changes
- * Never mix `async pipe` + manual subscribe on same Observable -- double subscription